



ANGULAR 20

TP 3 — Directives, Pipes & Introduction aux Services

📋 Compétences couvertes :

- 📋 Directives Structurelles — @if / @for / @switch
- 📋 Directives d'Attribut — ngClass, ngStyle
- 📋 Directives Personnalisées — @Directive, @HostListener, @HostBinding
- 📋 Pipes — Built-in (date, currency, uppercase...) + Custom
- 📋 Services & Injection de Dépendances — @Injectable, providedIn: 'root'
- 📋 Application complète — Gestionnaire de Contacts enrichi

📋 Objectifs pédagogiques

Ce TP vous permet de pratiquer les concepts clés de la séance 3 d'Angular 20 à travers 6 exercices progressifs. À la fin, vous serez capables de :

✓	Utiliser @if, @for, @switch pour contrôler l'affichage conditionnel et en boucle
✓	Appliquer [ngClass] et [ngStyle] pour modifier dynamiquement l'apparence
✓	Créer une directive personnalisée avec @Directive, @HostListener et @HostBinding
✓	Utiliser les pipes built-in : uppercase, date, currency, number, percent, slice
✓	Créer un pipe custom (initiales, mention) avec @Pipe et PipeTransform
✓	Créer un service avec @Injectable et l'injecter dans plusieurs composants

Commandes CLI — Nouveaux éléments à générer

```
# 1. (Optionnel) Créer un nouveau projet
ng new gestion-contacts-s3
cd gestion-contacts-s3

# 2. Générer les composants nécessaires
ng generate component liste-contacts
ng generate component formulaire-contact
ng generate component stats-contacts

# 3. Générer le service
ng generate service contact

# 4. Générer la directive personnalisée
ng generate directive survol

# 5. Générer les pipes personnalisés
ng generate pipe initiales
ng generate pipe mention

# 6. Lancer le serveur
ng serve
```

Structure du projet à obtenir

```
src/app/
├── app.component.ts           ← Composant racine
├── contact.service.ts        ← Service partagé (données)
├── contact.interface.ts      ← Interface Contact
├── liste-contacts/
│   ├── liste-contacts.component.ts ← @for, ngClass, directive, pipe
│   └── liste-contacts.component.html
├── formulaire-contact/
│   └── formulaire-contact.component.ts ← [(ngModel)], @Output
├── stats-contacts/
│   └── stats-contacts.component.ts ← ngStyle, pipes
├── survol.directive.ts       ← @HostListener
├── initiales.pipe.ts         ← Pipe custom
└── mention.pipe.ts           ← Pipe custom bonus
```

🔗 EXERCICE 1 — Directives Structurelles

Exercice 1 — Directives Structurelles 🕒 20 min

Objectif : Maîtriser @if, @for, @switch pour contrôler le DOM

Rappel théorique

Les directives structurelles MODIFIENT la structure du DOM (ajout / suppression d'éléments).

@if (≈ *ngIf) : affiche un bloc si la condition est vraie

@for (≈ *ngFor) : répète un bloc pour chaque élément d'un tableau

@switch (≈ *ngSwitch) : affiche le bloc correspondant à une valeur

Partie A — Interface Contact et données de démo

Créez le fichier src/app/contact.interface.ts :

```
// contact.interface.ts
export interface Contact {
  nom: string;
  email: string;
  actif: boolean;
  score: number; // 0-20
  role: 'admin' | 'user' | 'guest';
}
```

Dans liste-contacts.component.ts, définissez les données initiales :

```
// liste-contacts.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Contact } from '../contact.interface';

@Component({
  selector: 'app-liste-contacts',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './liste-contacts.component.html',
})
export class ListeContactsComponent implements OnInit {

  contacts: Contact[] = [];
  filtreActif: boolean | null = null;

  // Propriété calculée : retourne la liste filtrée
  get contactsFiltres(): Contact[] {
    if (this.filtreActif === null) return this.contacts;
    return this.contacts.filter(c => c.actif === this.filtreActif);
  }

  ngOnInit(): void {
    this.contacts = [
      { nom: 'Ahmed Benali', email: 'ahmed@ma', actif: true, score: 16, role: 'admin' },
    ]
  }
}
```

```

    { nom: 'Sara Alami', email: 'sara@ma', actif: false, score: 8, role: 'user' },
    { nom: 'Omar Tazi', email: 'omar@ma', actif: true, score: 18, role: 'admin' },
    { nom: 'Laila Rami', email: 'laila@ma', actif: true, score: 11, role: 'guest' },
    { nom: 'Youssef El', email: 'youssef@ma', actif: false, score: 6, role: 'user' },
  ];
}
}

```

Partie B — @if : Affichage conditionnel

Dans liste-contacts.component.html, implémentez les boutons de filtre et l'affichage conditionnel :

```

<!-- liste-contacts.component.html -->
<div>
  <h2>Contacts ({{ contacts.length }})</h2>

  <!-- Boutons filtre (Event Binding) -->
  <button (click)="filtreActif = null">Tous ({{ contacts.length }})</button>
  <button (click)="filtreActif = true">Actifs</button>
  <button (click)="filtreActif = false">Inactifs</button>

  <!-- @if : message si liste vide -->
  @if (contactsFiltres.length === 0) {
    <p>Aucun contact dans cette catégorie.</p>
  }

  <!-- @if / @else if / @else -->
  @if (contacts.length > 10) {
    <p>☑ Carnet bien rempli !</p>
  } @else if (contacts.length > 0) {
    <p>☑ {{ contacts.length }} contact(s) enregistré(s)</p>
  } @else {
    <p>☑ Le carnet est vide.</p>
  }
</div>

```

Partie C — @for : Boucle avec variables spéciales

Continuez le template pour afficher la liste avec @for :

```

<!-- @for avec variables $index, $first, $last, $even, $count -->
<ul>
  @for (c of contactsFiltres; track c.email; let i = $index) {
    <li>
      <!-- Numéro de ligne -->
      <span>{{ i + 1 }}</span>

      <!-- Nom et email -->
      <strong>{{ c.nom }}</strong> – {{ c.email }}

      <!-- Indicateur actif/inactif -->
      @if (c.actif) { <span>☑ Actif</span> }
      @else { <span>☑ Inactif</span> }

      <!-- Marqueur premier/dernier -->
      @if ($first) { <small> (Premier)</small> }
    }
  }

```

```

        @if ($last) { <small> (Dernier)</small> }
      </li>
    } @empty {
      <li>Aucun résultat pour ce filtre.</li>
    }
  </ul>

<!-- Bonus : @for avec $even/$odd pour lignes alternées -->
<table>
  @for (c of contacts; track c.email; let pair = $even) {
    <tr [style.background]="pair ? '#F5F5F5' : '#FFFFFF'">
      <td>{{ c.nom }}</td>
      <td>{{ c.score }}/20</td>
    </tr>
  }
</table>

```

Partie D — @switch : Rôle des contacts

Ajoutez un affichage conditionnel selon le rôle de chaque contact dans la boucle :

```

<!-- Dans le @for, après le nom -->
@switch (c.role) {
  @case ('admin') {
    <span>👤 Admin</span>
  }
  @case ('user') {
    <span>👤 Utilisateur</span>
  }
  @case ('guest') {
    <span>👤 Visiteur</span>
  }
  @default {
    <span>? Rôle inconnu</span>
  }
}

```

✔ Résultat attendu

Les boutons Tous / Actifs / Inactifs filtrent dynamiquement la liste.
Chaque ligne affiche le numéro, le nom, l'état actif/inactif et le rôle.
Le bloc @empty s'affiche si la liste filtrée est vide.

Questions de compréhension (1)

1. Quelle est la différence entre @if (condition) { } et *ngIf dans le template ?
2. Pourquoi faut-il obligatoirement track dans @for ? Quel problème évite-t-on ?
3. Quelles sont les 6 variables spéciales disponibles dans @for ?
4. Peut-on imbriquer un @if dans un @for ? Et un @for dans un @if ?

EXERCICE 2 — Directives d'Attribut : ngClass & ngStyle

Exercice 2 — Directives d'Attribut 20 min

Objectif : Modifier dynamiquement l'apparence avec ngClass et ngStyle

Rappel théorique

[ngClass] : ajoute/supprime des classes CSS selon des conditions (objet, tableau ou expression).

[ngStyle] : applique des styles CSS inline dynamiques (objet clé-valeur).

Règle : préférez ngClass quand les valeurs sont dans un fichier CSS — ngStyle pour des valeurs calculées.

Partie A — ngClass sur les lignes de la liste

Ajoutez du CSS dans liste-contacts.component.css :

```
/* liste-contacts.component.css */
.contact-actif { background: #E8F5E9; border-left: 4px solid #4CAF50; }
.contact-inactif { background: #FFEBEE; border-left: 4px solid #F44336; color: #9E9E9E; }
.top-score { font-weight: bold; }
.score-excellent { color: #1565C0; }
.score-passable { color: #E65100; }
.score-echec { color: #B71C1C; text-decoration: line-through; }
.premier-item { border-top: 2px solid #673AB7; }
```

Modifiez la boucle @for pour appliquer ngClass :

```
<!-- Forme 1 : objet { 'classe': condition } -->
<li [ngClass]="{
  'contact-actif': c.aktif,
  'contact-inactif': !c.aktif,
  'top-score': c.score >= 16,
  'premier-item': $first
}">
  {{ c.nom }} — Score : {{ c.score }}/20
</li>

<!-- Forme 2 : expression de classe concaténée -->
<span [ngClass]="score-' + (c.score >= 16 ? 'excellent' :
  c.score >= 10 ? 'passable' : 'echec')">
  {{ c.score }}/20
</span>
```

Partie B — ngStyle pour une barre de score dynamique

Ajoutez un composant de statistiques stats-contacts. Dans stats-contacts.component.ts :

```
// stats-contacts.component.ts
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Contact } from '../contact.interface';
```

```

@Component({
  selector: 'app-stats-contacts',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './stats-contacts.component.html',
})
export class StatsContactsComponent {
  @Input() contacts: Contact[] = [];

  get totalActifs(): number {
    return this.contacts.filter(c => c.actif).length;
  }

  get tauxActivite(): number {
    if (this.contacts.length === 0) return 0;
    return Math.round((this.totalActifs / this.contacts.length) * 100);
  }

  get scoreMoyen(): number {
    if (this.contacts.length === 0) return 0;
    const total = this.contacts.reduce((sum, c) => sum + c.score, 0);
    return Math.round(total / this.contacts.length);
  }

  // Couleur dynamique selon le taux d'activité
  get couleurBarre(): string {
    if (this.tauxActivite >= 70) return '#4CAF50';
    if (this.tauxActivite >= 40) return '#FF9800';
    return '#F44336';
  }
}

```

Template avec ngStyle

```

<!-- stats-contacts.component.html -->
<div>
  <h3>Statistiques</h3>

  <p>Total : {{ contacts.length }}</p>
  <p>Actifs : {{ totalActifs }} / {{ contacts.length }}</p>
  <p>Score moyen : {{ scoreMoyen }}/20</p>

  <!-- Barre de progression avec ngStyle -->
  <div [ngStyle]="{ 'width': '100%', 'background': '#E0E0E0',
    'border-radius': '4px', 'height': '20px' }">
    <div [ngStyle]="{
      'width':          tauxActivite + '%',
      'background-color': couleurBarre,
      'height':         '20px',
      'border-radius':  '4px',
      'transition':     'width 0.5s ease'
    }">
    </div>
  </div>
  <small>Taux d'activité : {{ tauxActivite }}%</small>

  <!-- Texte dont la couleur change selon le score moyen -->
  <p [ngStyle]="{

```

```
'color':      scoreMoyen >= 14 ? '#1B5E20' : '#B71C1C',  
'font-weight': 'bold',  
'font-size':  (16 + scoreMoyen / 4) + 'px'  
}"]>  
  Score moyen : {{ scoreMoyen }}/20  
</p>  
</div>
```

✓ Résultat attendu

Les lignes actives sont vertes, les inactives rouges (ngClass).

La barre de progression se remplit proportionnellement au taux d'activité.

La couleur de la barre change du rouge au vert selon le pourcentage.

Questions de compréhension (2)

1. Quelle est la différence entre `[class.active]="condition"` et `[ngClass]="{'active': condition}"` ?
2. Pourquoi `ngStyle` est-il préféré à `[style.xxx]` quand on veut modifier plusieurs propriétés CSS ?
3. Comment faire en sorte qu'une classe soit appliquée uniquement sur les lignes paires ?

EXERCICE 3 — Directive Personnalisée

Exercice 3 — Directive de survol ⌚ 15 min

Objectif : Créer une directive réutilisable avec @HostListener et @HostBinding

Rappel théorique

@Directive : décorateur qui marque la classe comme directive Angular

@HostListener : écoute un événement DOM sur l'élément hôte (ex: mouseenter, click)

@HostBinding : lie une propriété de l'élément hôte (class, style, attr...) à une variable

ElementRef : référence directe à l'élément DOM natif

Partie A — Directive de survol avec ElementRef

Contenu du fichier survol.directive.ts :

```
// survol.directive.ts
import { Directive, ElementRef,
        HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appSurvol]', // Selecteur attribut HTML
  standalone: true
})
export class SurvolDirective {

  // Paramètre reçu depuis le template
  @Input() appSurvol: string = '#FFFDE7'; // Jaune pâle par défaut

  private couleurOriginale: string = '';

  constructor(private el: ElementRef) {}

  // Événement mouseenter : la souris entre sur l'élément
  @HostListener('mouseenter')
  onMouseEnter(): void {
    // Sauvegarder la couleur originale
    this.couleurOriginale =
      this.el.nativeElement.style.backgroundColor;
    // Appliquer la nouvelle couleur
    this.el.nativeElement.style.backgroundColor = this.appSurvol;
    this.el.nativeElement.style.cursor = 'pointer';
  }

  // Événement mouseleave : la souris quitte l'élément
  @HostListener('mouseleave')
  onMouseLeave(): void {
    // Restaurer la couleur originale
    this.el.nativeElement.style.backgroundColor = this.couleurOriginale;
  }
}
```

Partie B — Directive avec @HostBinding (variante déclarative)

Créez une deuxième directive `highlight.directive.ts` utilisant `@HostBinding` :

```
// highlight.directive.ts
import { Directive, HostBinding,
  HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective {

  @Input() isActive: boolean = false;

  // @HostBinding lie la classe CSS 'active' à la propriété isActive
  @HostBinding('class.active')
  get active() { return this.isActive; }

  // @HostBinding lie la classe CSS 'hovered' à isHovered
  @HostBinding('class.hovered')
  isHovered: boolean = false;

  // @HostBinding pour le style inline
  @HostBinding('style.box-shadow')
  get shadow(): string {
    return this.isHovered ? '0 2px 8px rgba(0,0,0,0.2)' : 'none';
  }

  @HostListener('mouseenter')
  onEnter() { this.isHovered = true; }

  @HostListener('mouseleave')
  onLeave() { this.isHovered = false; }
}
```

Partie C — Utiliser les directives dans le template

Dans `liste-contacts.component.ts`, importez les directives :

```
// Ajouter dans les imports du composant
import { SurvolDirective } from '../survol.directive';
import { HighlightDirective } from '../highlight.directive';

@Component({
  standalone: true,
  imports: [CommonModule, SurvolDirective, HighlightDirective],
  ...
})
```

Dans le template, appliquez les directives :

```
<!-- Survol jaune par défaut -->
<li appSurvol>{{ c.nom }}</li>

<!-- Survol avec couleur personnalisée -->
<li [appSurvol]="lightblue">{{ c.nom }}</li>
```

```
<!-- Couleur selon le rôle du contact -->
<li [appSurvol]="c.role === 'admin' ? '#FFE0E0' : '#E0F0FF'">
  {{ c.nom }} ({{ c.role }})
</li>

<!-- Directive @HostBinding -->
<div appHighlight [isActive]="c.aktif">
  Élément actif qui réagit au survol
</div>
```

📌 CSS nécessaire pour @HostBinding

Ajoutez dans styles.css :

```
.active { border: 2px solid #4CAF50; }
.hovered { background: #FFF9C4; }
```

Questions de compréhension (3)

1. Quelle est la différence entre @HostListener et (click) dans un template ?
2. Pourquoi utiliser @HostBinding plutôt que el.nativeElement.style directement ?
3. Comment passer plusieurs paramètres à une directive via @Input() ?
4. Peut-on appliquer plusieurs directives sur le même élément HTML ?

🔗 EXERCICE 4 — Pipes Built-in & Personnalisés

Exercice 4 — Pipes 🕒 20 min

Objectif : Transformer l'affichage avec les pipes intégrés et créer ses propres pipes

Tableau de référence — Pipes built-in Angular

Pipe	Syntaxe	Résultat	Description
uppercase	{{ 'bonjour' uppercase }}	'BONJOUR'	Majuscules
lowercase	{{ 'BONJOUR' lowercase }}	'bonjour'	Minuscules
titlecase	{{ 'bonjour monde' titlecase }}	'Bonjour Monde'	1ère lettre de chaque mot
date	{{ today date:'dd/MM/yyyy' }}	'14/04/2025'	Format date
currency	{{ 1500 currency:'MAD' }}	'MAD 1,500.00'	Montant + devise
number	{{ 3.14159 number:'1.2-2' }}	'3.14'	Format numérique
percent	{{ 0.85 percent }}	'85%'	Pourcentage
json	{{ objet json }}	JSON formaté	Debug d'objets
slice	{{ [1,2,3,4] slice:1:3 }}	[2, 3]	Extraire portion

Partie A — Utiliser les pipes built-in dans le template

Dans liste-contacts.component.html, enrichissez chaque ligne avec des pipes :

```
@for (c of contacts; track c.email; let i = $index) {
  <li>
    <!-- Numéro -->
    {{ i + 1 }}.

    <!-- uppercase : nom en majuscules -->
    <strong>{{ c.nom | uppercase }}</strong>

    <!-- slice : afficher seulement les 20 premiers caractères de l'email -->
    <em>{{ c.email | slice:0:20 }}</em>

    <!-- Score avec number : toujours 1 décimale -->
    Score : {{ c.score | number:'1.1-1' }}/20

    <!-- Taux de réussite en pourcentage -->
    ({{ c.score / 20 | percent:'1.0-0' }})

    <!-- Date d'inscription (simulée) -->
    {{ today | date:'dd/MM/yyyy' }}

    <!-- Chaînage de pipes -->
    Initiales : {{ c.nom | uppercase | slice:0:1 }}
  </li>
}
```

}

Partie B — Créer le pipe 'initiales'

Contenu du fichier initiales.pipe.ts :

```
// initiales.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'initiales',      // Nom utilisé dans le template
  standalone: true
})
export class InitialesPipe implements PipeTransform {

  // valeur    : la chaîne à transformer
  // sep       : séparateur optionnel (défaut '.')
  transform(valeur: string, sep: string = '.'): string {
    if (!valeur || valeur.trim() === '') return '';

    return valeur
      .trim()
      .split(' ')                // ['Ahmed', 'Benali']
      .filter(mot => mot.length > 0) // Ignorer espaces doubles
      .map(mot => mot[0].toUpperCase()) // ['A', 'B']
      .join(sep);                // 'A.B'
  }
}
```

Utilisation dans le template (après import dans imports:[]) :

```
<!-- Import : InitialesPipe dans imports: [] du composant -->

<!-- Séparateur par défaut -->
{{ c.nom | initiales }}
<!-- Ahmed Benali --> 'A.B'

<!-- Séparateur personnalisé -->
{{ c.nom | initiales:'-' }}
<!-- Ahmed Benali --> 'A-B'

<!-- Chaînage avec uppercase -->
{{ c.nom | initiales | uppercase }}
<!-- 'A.B' (déjà en majuscules, mais démontre le chaînage) -->
```

Partie C — Pipe 'mention' (Exercice Bonus)

Créez mention.pipe.ts qui retourne la mention selon le score :

```
// mention.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'mention', standalone: true })
export class MentionPipe implements PipeTransform {
```

```

transform(score: number): string {
  if (score >= 18) return '🏆 Félicité';
  if (score >= 16) return '🏆 Très Bien';
  if (score >= 14) return '🏆 Bien';
  if (score >= 10) return '✅ Passable';
  return '❌ Ajourné';
}

```

Utilisation dans le template :

```

<!-- Afficher la mention de chaque contact -->
{{ c.score | mention }}

```

```

<!-- Exemples de résultats -->
18 | mention --> '🏆 Félicité'
16 | mention --> '🏆 Très Bien'
14 | mention --> '🏆 Bien'
11 | mention --> '✅ Passable'
8  | mention --> '❌ Ajourné'

```

✅ Résultat attendu

Chaque contact affiche ses initiales (A.B), le score formaté, le pourcentage et la mention.

Les pipes sont chaînés de gauche à droite.

La valeur originale dans le composant n'est jamais modifiée par le pipe.

Questions de compréhension (4)

1. Un pipe modifie-t-il la valeur originale dans le composant ? Pourquoi ?
2. Quelle est la différence entre un pipe pur et un pipe impure en Angular ?
3. Comment passer plusieurs paramètres à un pipe ? Donnez un exemple.
4. Peut-on utiliser un pipe dans une condition @if ? Comment ?

📌 EXERCICE 5 — Services & Injection de Dépendances

Exercice 5 — ContactService 🕒 20 min

Objectif : Créer un service partagé et l'injecter dans plusieurs composants

Rappel théorique

@Injectable({ providedIn: 'root' }): Le service est un singleton dans toute l'application.

Injection : déclarez le service dans le constructeur du composant — Angular crée et injecte l'instance.

Principe SRP : le composant gère l'AFFICHAGE, le service gère les DONNÉES et la LOGIQUE.

Partie A — Créer le ContactService

Contenu de contact.service.ts :

```
// contact.service.ts
import { Injectable } from '@angular/core';
import { Contact } from './contact.interface';

@Injectable({
  providedIn: 'root' // Singleton : une seule instance dans toute l'app
})
export class ContactService {

  // Données privées (source de vérité unique)
  private contacts: Contact[] = [
    { nom: 'Ahmed Benali', email: 'ahmed@ma', actif: true, score: 16, role: 'admin' },
    { nom: 'Sara Alami', email: 'sara@ma', actif: false, score: 8, role: 'user' },
    { nom: 'Omar Tazi', email: 'omar@ma', actif: true, score: 18, role: 'admin' },
    { nom: 'Laila Rami', email: 'laila@ma', actif: true, score: 11, role: 'guest' },
  ];

  // — LECTURE —
  getAll(): Contact[] {
    return this.contacts;
  }

  getActifs(): Contact[] {
    return this.contacts.filter(c => c.actif);
  }

  getByRole(role: string): Contact[] {
    return this.contacts.filter(c => c.role === role);
  }

  getScoreMoyen(): number {
    if (this.contacts.length === 0) return 0;
    return Math.round(
      this.contacts.reduce((s, c) => s + c.score, 0) / this.contacts.length
    );
  }

  // — MODIFICATION —
```

```

ajouter(contact: Contact): void {
  this.contacts.push(contact);
}

supprimer(email: string): void {
  this.contacts = this.contacts.filter(c => c.email !== email);
}

toggleActif(email: string): void {
  const contact = this.contacts.find(c => c.email === email);
  if (contact) contact.actif = !contact.actif;
}
}

```

Partie B — Injecter le service dans ListeContactsComponent

Mettez à jour liste-contacts.component.ts pour utiliser le service :

```

// liste-contacts.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ContactService } from '../contact.service';
import { Contact } from '../contact.interface';
import { SurvolDirective } from '../survol.directive';
import { InitialesPipe } from '../initiales.pipe';
import { MentionPipe } from '../mention.pipe';

@Component({
  selector: 'app-liste-contacts',
  standalone: true,
  imports: [CommonModule, SurvolDirective, InitialesPipe, MentionPipe],
  templateUrl: './liste-contacts.component.html',
})
export class ListeContactsComponent implements OnInit {

  contacts: Contact[] = [];

  // Angular INJECTE automatiquement une instance de ContactService
  constructor(private contactService: ContactService) {}

  ngOnInit(): void {
    // Charger les données depuis le service
    this.contacts = this.contactService.getAll();
  }

  supprimer(email: string): void {
    this.contactService.supprimer(email);
    // Rafraîchir la liste
    this.contacts = this.contactService.getAll();
  }

  toggleActif(email: string): void {
    this.contactService.toggleActif(email);
    this.contacts = this.contactService.getAll();
  }
}

```


Partie C — Injecter le même service dans StatsContactsComponent

Démontrez que le service est un singleton en injectant le même service dans StatsContactsComponent :

```
// stats-contacts.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ContactService } from '../contact.service';

@Component({
  selector: 'app-stats-contacts',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './stats-contacts.component.html',
})
export class StatsContactsComponent implements OnInit {

  totalContacts: number = 0;
  totalActifs: number = 0;
  scoreMoyen: number = 0;

  // Le MEME service est injecté – même instance singleton
  constructor(private contactService: ContactService) {}

  ngOnInit(): void {
    this.totalContacts = this.contactService.getAll().length;
    this.totalActifs = this.contactService.getActifs().length;
    this.scoreMoyen = this.contactService.getScoreMoyen();
  }
}
```

📖 Concept clé — Singleton

Quand ListeContactsComponent et StatsContactsComponent injectent ContactService, Angular leur fournit LA MÊME instance (singleton).

Si ListeContacts supprime un contact, StatsContacts verra le changement après refresh. C'est le principe de 'source de vérité unique' (Single Source of Truth).

Questions de compréhension (5)

1. Pourquoi utilise-t-on private dans constructor(private contactService: ContactService) ?
2. Que se passerait-il si on avait TWO instances du ContactService ? Quel problème ?
3. Quelle est la différence entre providedIn: 'root' et providedIn: 'any' ?
4. Peut-on injecter un service dans un autre service ? Comment ?

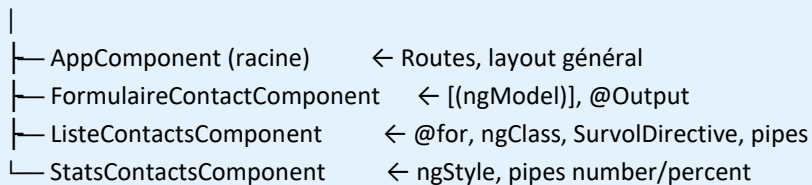
EXERCICE 6 — Application Complète

Exercice 6 — Assemblage final 15 min

Objectif : Assembler tous les concepts dans une application unifiée

Architecture finale de l'application

ContactService (singleton) — getAll(), ajouter(), supprimer(), toggleActif()



Directives : SurvolDirective, HighlightDirective

Pipes : InitialesPipe, MentionPipe (+ built-in : uppercase, date, currency, percent)

AppComponent — Assemblage général

```

// app.component.ts
import { Component, OnInit } from '@angular/core';
import { ContactService } from './contact.service';
import { Contact } from './contact.interface';
import { ListeContactsComponent }
  from './liste-contacts/liste-contacts.component';
import { FormulaireContactComponent }
  from './formulaire-contact/formulaire-contact.component';
import { StatsContactsComponent }
  from './stats-contacts/stats-contacts.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [ListeContactsComponent,
    FormulaireContactComponent,
    StatsContactsComponent],
  template: `
    <h1>📋 Gestionnaire de Contacts Angular 20</h1>

    <!-- Statistiques en haut -->
    <app-stats-contacts></app-stats-contacts>

    <!-- Formulaire d'ajout -->
    <app-formulaire-contact
      (contactAjoute)="onContactAjoute($event)">
    </app-formulaire-contact>

    <!-- Liste des contacts -->
    <app-liste-contacts></app-liste-contacts>
  `
})
export class AppComponent implements OnInit {

```

```

constructor(private contactService: ContactService) {}

ngOnInit(): void {
  console.log('App initialisée. Contacts :', this.contactService.getAll().length);
}

onContactAjoute(contact: Contact): void {
  this.contactService.ajouter(contact);
}
}

```

Checklist de validation finale

✓	Critère de validation
☐	Les directives @if, @for, @switch sont utilisées dans le template
☐	@for inclut track pour les performances + @empty pour la liste vide
☐	[ngClass] applique des classes CSS différentes selon actif/inactif
☐	[ngStyle] anime une barre de progression dynamique dans les stats
☐	La directive appSurvol change la couleur de fond au survol de la souris
☐	La directive appHighlight utilise @HostBinding pour les classes CSS
☐	Le pipe initiales affiche les initiales de chaque contact
☐	Le pipe mention affiche la mention selon le score
☐	Les pipes built-in (uppercase, date, currency, percent) sont utilisés
☐	ContactService est injecté dans au moins 2 composants différents
☐	Les méthodes de logique (filtre, calculs) sont dans le service, pas dans le composant
☐	La console F12 ne montre aucune erreur Angular

🔍 Erreurs fréquentes & Solutions

✖ Erreur	🔍 Cause	✅ Solution
ngClass / ngStyle ne s'appliquent pas	CommonModule non importé	Ajouter CommonModule dans imports: []
Pipe inconnu dans le template	Pipe non importé	Ajouter MonPipe dans imports: [] du composant
'track' manquant dans @for	Propriété track absente	@for (item of list; track item.id)
Directive ignorée sur l'élément	Directive non importée	Ajouter MaDirective dans imports: []
Service différent selon composant	Deux providedIn différents	Utiliser providedIn: 'root' partout
Données non mises à jour après .push()	Référence identique	Créer nouveau tableau : [...tab, item]
@HostBinding classe non appliquée	CSS absent dans styles.css	Définir .maClasse { } dans styles.css
transform() non appelé	Interface PipeTransform absente	implements PipeTransform